European Conference on Computational Biology, 2026, pp. 1–7
DOI added during production
Published: Date added during production
Paper

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum,[1,2,*] Bart Buelens,[1] Ruben Taelman[2] and Gökhan Ertaylan[1]

[2]IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium
[1]Digital Biological Systems, Flemish institute for Technological Research (VITO), Boeretang 200, 2400, Belgium

[*]Corresponding author. E-mail: elias.crum@ugent.be

Abstract

The Variant Call Format (VCF) is the dominant standard for representing genomic variation and underpins large-scale population genomics, clinical sequencing, and precision medicine workflows. As the use of VCF data expands, there is increasing interest in enriching variant datasets with complementary information such as sample metadata, clinical test results, links to public biomedical resources, and privacy-related annotations. To accommodate the structured and scalable linkage of these heterogeneous data, we see promise in using a knowledge graph representation via the Resource Description Framework (RDF). To benefit from the potential of RDF, the initial challenge is translating VCF file data into RDF. Here, we present VCF-RDFizer, a command-line tool available on PyPI and Conda-Forge that provides an end-to-end pipeline for converting VCF into RDF. VCF-RDFizer introduces a publicly available vocabulary that models VCF constructs while integrating established ontologies from biomedical graphs. The tool preserves variant-level detail, genotype information, and metadata, enabling an enriched and standards-aware representation of genomic variation. Unlike ad hoc or partial transformations, VCF-RDFizer offers a systematic and reproducible end-to-end workflow from VCF to structured RDF in raw N-Triples and/or compressed formats. To assess the feasibility of our tool, we tested VCF-RDFizer on six publicly available VCF files of varying sizes and complexities. Our evaluation demonstrates that conversion is tractable across realistic dataset scales and can be performed successfully on workstation-class hardware. By transforming VCF into an interoperable knowledge graph format, VCF-RDFizer lays the foundation for the exciting possibilities of heterogeneous data enrichment of VCF.

Keywords Knowledge Representation, Genomic Variant Data, Variant Call Format, Knowledge Graphs, Semantics

Introduction

Variant Call Format. The Variant Call Format (VCF) is the dominant standard for representing and exchanging genomic variants across research and clinical sequencing pipelines, including pharmacogenomics and precision medicine applications (2; 22; 28). As clinical sequencing expands, variant datasets are increasingly interpreted alongside external biomedical resources, patient or sample metadata, and governance constraints controlling access to sensitive genomic information. Operational semantic-genomics initiatives already demonstrate the value of linking variant observations to curated biomedical databases and controlled-access frameworks (25; 30).

These emerging use cases expose limitations of the traditional VCF representation. First, clinical interpretation frequently requires linking variants to external knowledge sources such as protein annotations, disease databases, or pharmacogenomic resources. While VCF supports annotation fields, it provides limited mechanisms for structured, machine-interpretable linkage between variant records and external resources. Second, genomic datasets frequently require fine-grained privacy controls, such as restricting access to particular genomic regions or subsets of variants, which the VCF specification does not natively support. Third, querying variant collections across multiple VCF files, for example, identifying clinically relevant mutations or retrieving variants within genomic position intervals, typically relies on specialized command-line utilities (e.g., bcftools (14)) or custom pipelines rather than interoperable, declarative query mechanisms.

In other domains, semantic web and policy frameworks, such as Linked Data (33) and the Open Digital Rights Language (ODRL) (20), enable structured linkage between datasets, external knowledge resources, and access control policies. Applying similar approaches to genomic variant data could enable richer integration with biomedical knowledge bases and more expressive query capabilities.

Resource Description Format. The Resource Description Framework (RDF) (13) provides a widely adopted graph-based data model for representing relationships between entities identified by globally unique identifiers (IRIs). For variant data, RDF enables (i) structured linkage between variant data and external biomedical resources (8; 12; 27), (ii) native declarative querying via the SPARQL query language (19), and (iii) fine-grained privacy policy enforcement over variant data and linkages (20). In practice, this could enable, for instance, a user to query a collection of VCF-derived datasets to identify variants whose alternative alleles occur in a specific gene,

Received: 16 March 2026. **Revised received:** Date Month Year. **Accepted:** Date Month Year
© The Author(s) 2026. Copyright and licence statements to be updated by the publisher during production.

Tool/workflow	Code language	CLI	Public vocab.	Reprod. pipeline	Metadata + genotype	Modular mappings	Compression pipeline	Public packaging
VCF-RDFizer (this study)	**Python**	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**
VCF2RDF (TOGOvar) (25)	Rust	Y	N	P	P (sub.)	N	N	N
JVarKit VcfToRdf (23)	Java	Y	N	P	P (sub.)	N	N	Y
ruby-rdf/rdf-vcf (9)	Ruby	P	N	P	P (sub.)	N	N	Y
Semantic beacons workflow (10)	YAML	N	Y	P	P (sub.)	N	N	N

Table 1 Comparison of prior VCF-to-RDF tooling and workflows against engineering-completeness criteria considered in this study. Columns indicate whether a system provides: a command-line interface (CLI) for direct pipeline use; a publicly documented semantic vocabulary describing the RDF model (Public vocab.); an end-to-end reproducible transformation workflow (Reprod. pipeline); support for representing VCF metadata, genotype, and sample-level information (Metadata + genotype); modular and user-extensible transformation rules (Modular mappings); integrated strategies for mitigating RDF storage expansion through compression (Compression pipeline); and publicly distributed software packaging for reuse (Public packaging). Status codes: Y = explicitly supported, P = partially supported, N = not supported. Inline qualifier: sub. = subset-level support.

are associated with a given phenotype, and are accessible under particular policy restrictions.

From VCF to RDF. Transforming VCF into RDF requires *semantic lifting*: converting the row-based structure of VCF files into graph statements. Relevant VCF elements such as genomic position, alleles, genotypes, and sample identifiers are assigned IRIs, and fields are translated into subject–predicate–object triples. Ontology-aligned predicates provide consistent semantics which enable datasets to be queried and linked.

This transformation requires two components: (i) a semantic vocabulary describing VCF concepts and relationships, and (ii) a mapping process that converts VCF file content into RDF using this vocabulary. Such mappings must be systematic and reproducible due to the structural heterogeneity permitted by the VCF specification (2). INFO fields may contain semi-structured annotations, FORMAT and genotype columns vary across datasets, and multi-sample files aggregate multiple genotypes within shared variant records, complicating the creation of consistent semantic representations.

Several efforts have explored representing VCF data in RDF. Table 1 compares the capabilities of existing tools including *VCF2RDF* within TOGOvar (25), JVarkit's *VcfToRdf* (23), the *ruby-rdf/rdf-vcf* library (9), and the *semantic beacons* workflow (10). While these systems demonstrate the feasibility of semantic representations of genomic variants, several limitations remain.

Related Work. Several existing efforts demonstrate that VCF data can be represented in RDF. Table 1 summarizes the capabilities of the most advanced VCF-to-RDF tools available at the time of writing, including *VCF2RDF* within TOGOvar (25), JVarkit's *VcfToRdf* (23), the *ruby-rdf/rdf-vcf* library (9), and the *semantic beacons* workflow (10). In terms of access channels, VCF-RDFizer is distributed through PyPI, Conda-Forge, and DockerHub, JVarkit is available as a JAR utility, ruby-rdf/rdf-vcf is available as a Ruby gem and GitHub repository, while TOGOvar and semantic-beacons implementations are primarily accessed as platform/workflow resources rather than standalone packaged conversion CLIs. While these systems demonstrate the feasibility of semantic representations of genomic variants, Table 1 highlights several capabilities that remain only partially addressed across existing tools.

Open Challenges. *First*, most approaches lack a publicly documented vocabulary describing the structural semantics of VCF data. As shown in Table 1, only the semantic beacons workflow publishes a vocabulary, and it covers only a subset of VCF fields. Although genomic ontologies such as GA4GH Variant Representation (31), FALDO (11), Sequence Ontology (17), GENO, and SNPO describe biological entities, they do not define a reusable semantic model of VCF file structure.

Second, many converters rely on hard-coded parsing logic or ad-hoc transformation rules. For example, the semantic beacons workflow uses YAML-based rule specifications (4). Declarative mapping languages provide a more robust alternative. RML (RDF Mapping Language) (15) supports mapping semi-structured and heterogeneous data sources to RDF and is therefore well-suited to the irregular structure of VCF files. Its ecosystem includes engines such as RMLStreamer (26) for scalable transformation workflows. By separating semantic mappings from implementation logic, RML enables modular rules that can be shared, adapted, and extended for dataset-specific structures.

Third, Table 1 shows that most tools do not address the storage implications of semantic lifting. Expanding compact formats such as VCF into explicit RDF triples increases storage size, particularly when using globally unique IRIs (13). Compression strategies therefore, play an important role. General-purpose compressors such as gzip and Brotli (5) can reduce RDF size substantially, while HDT (Header–Dictionary–Triples) (18) provides a compressed RDF representation that supports direct querying.

Study Aims. Taken together, these limitations motivate the development of a reusable, reproducible VCF-to-RDF transformation tool. Here we present *VCF-RDFizer*, which integrates a publicly documented vocabulary, modular RML-based mappings, and compression-aware transformation pipelines. As summarized in Table 1, VCF-RDFizer combines capabilities that are typically disconnected in existing approaches: a standalone CLI `https://github.com/ecrum19/VCF-RDFizer`, a public vocabulary `https://w3id.org/vcf-rdfizer/vocab#`, modular declarative mappings, a reproducible pipeline, and public packaging `https://pypi.org/project/vcf-rdfizer/`.

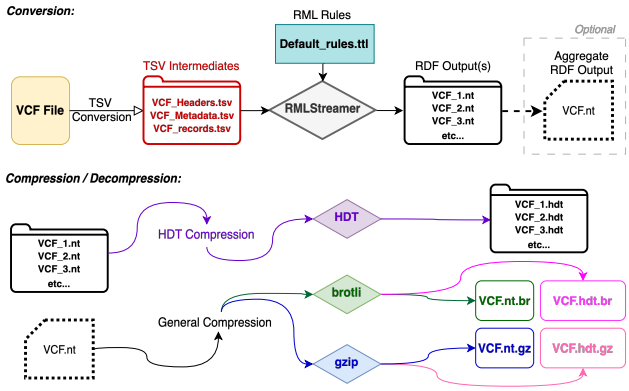

Figure 1 VCF-RDFizer Pipeline Workflow The flowchart depicts end-to-end conversion from VCF input to semantically enriched, compressed RDF outputs. **Conversion**: VCF-RDFizer converts VCF file into TSV intermediates, uses RML mappings and RMLStreamer to generate N-Triples, and optionally aggregates VCF RDF outputs to a single N-Triples ('.nt') file. **Compression**: VCF-RDFizer compresses RDF output(s) via HDT and/or General Compression (gzip, Brotli) for downstream storage and querying. HDT compressed RDF files can also be further compressed via general compression (either gzip or Brotli). **Decompression**: All steps in the compression workflow are reversible.

VCF-RDFizer CLI Tool

VCF-RDFizer is a command-line tool that converts input VCF files into semantically enriched RDF using a reproducible mapping pipeline. The workflow integrates pre-processing, mapping, RDF conversion, and optional compression/decompression functionalities. An overview of these workflows is shown in Figure 1.

Semantic Model

The semantic model we developed to represent VCF data combines a dedicated VCF-RDFizer vocabulary (`https://w3id.org/vcf-rdfizer/vocab#`) with terms aligned with established biological ontologies to maximize interoperability and reuse. Term equivalencies are defined for genomic coordinates (FALDO (11)), variant typing (the Sequence Ontology (17)), genotype/sample constructs (HERO (24)), and features of the GA4GH VRS schema v2.0.1 (32). This hybrid strategy retains VCF-specific semantics while avoiding duplication of mature ontology terms.

The model preserves three information layers: metadata/header assertions, variant-level assertions, and genotype/sample assertions. Core entities are minted using deterministic IRI templates derived from stable VCF attributes (e.g., chromosome, position, reference allele, etc.), which keeps identifiers reproducible across runs. The vocabulary website also provides a suggested canonical IRI pattern for instances of the core classes, which can be used as a reference for users who want to generate their own RDF from VCF files using the provided vocabulary and mappings. A visual representation of the core components of the vocabulary is shown in Figure 2, and a more detailed and interactive version is available on the companion website: `https://ecrum19.github.io/VCF-RDFizer-vocabulary/`. On the website are also examples of how a VCF file will look after conversion to RDF (as well as an example for multi-sample cases).

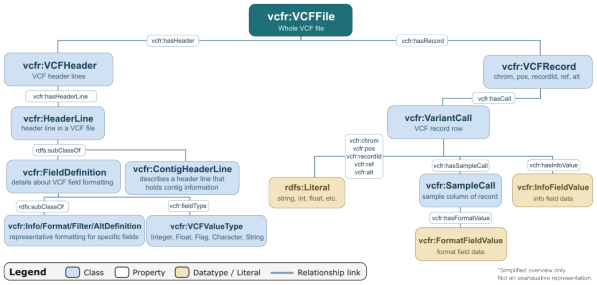

Figure 2 VCF-RDFizer Vocabulary Semantic Model (simplified) The figure depicts a simplified representation of the semantic hierarchy of core terms of the VCF-RDFizer vocabulary. All *vcfr:* prefixes are an abbreviation for `https://w3id.org/vcf-rdfizer/vocab#`. Genotype/sample class relationships are not represented in this figure for simplicity, but more details are available in the interactive representation of the vocabulary at: `https://ecrum19.github.io/VCF-RDFizer-vocabulary/`.

Tool Workflow

VCF-RDFizer is implemented as Python tool that mediates four main pipeline stages: (1) VCF conversion to TSV, (2) TSV conversion to RDF via RML mappings and RMLStreamer, (3) RDF output aggregation (optional), and (4) result compression (optional) and run statistics outputs. Outputs are emitted as N-Triples by default, with various optional arguments to mediate knowledge graph output formatting and/or compression.

Mapping rules are authored in Turtle as modular RML rule sets that mediate the conversion from VCF to RDF. This modularity allows rules to be extended or modified without rewriting the full pipeline.

To execute conversion using RML mappings, we utilized a Java-based tool from the RML ecosystem: RMLStreamer (26). RMLStreamer (version 2.5.0) was executed using its standalone command-line interface (RMLStreamer-v2.5.0-standalone.jar) with default Java heap settings.

Compression formats offered: gzip-compression (via the standard Linux gzip utility), brotli-compression (5) (version v1.1.0), and HDT, via the HDT-cpp command-line tool (18) (version v1.3.3). For all transformations, the tool logs performance indicators (wall time, input size, output size, and maximum resident set size (max_RSS)) to enable direct comparison of compression effectiveness and computational overhead. We used a Brotli compression level of 7 out of 11 because preliminary experiments with a higher compression level introduced higher run times and minimal compression ratio improvement.

The implementation and default RML mappings are available at `https://github.com/ecrum19/VCF-RDFizer`.

From a user-experience perspective, each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and DockerHub so users can choose pip-, conda-, or container-based installation with matched versions. The GitHub README serves as the primary user guide, including quick-start commands, end-to-end usage examples, command-line option documentation, and troubleshooting notes. For maintainability, the project follows a release-oriented workflow with versioned changelogs, public issue triage, planned periodic dependency/container updates, and extensive unit-testing with the tool to preserve reproducibility over time.

Input VCF	Input .vcf.gz size (MB)	Wall time (s)	Max RSS (MB)	Output raw RDF size (MB)	# triples
0GOOR_HG002	387.9	621.0	1,602.4	31,181.3	167,958,298
60820188474283	1,054.8	719.0	1,674.0	35,408.1	185,443,019
NG131FQA1I	1,215.5	630.7	2,027.3	25,969.3	147,710,127
60820188475559	1,233.0	872.2	1,546.3	37,955.6	198,039,339
NB72462M	1,337.0	700.2	1,878.5	26,764.6	155,532,530
NG1N86S6FC	2,076.8	1,693.5	1,943.2	77,729.7	457,412,987

Table 2 Conversion statistics per input file. Sizes are decimal MB.

Tool Testing

VCF Test Data To test our workflow, we used one publicly available VCF file from the NIST Genome in a Bottle project – 0GOOR_HG002 (3) and 5 publicly available VCF files from the Personal Genome Project (1). The selected files vary in size, record counts, and annotation patterns, and were chosen to stress realistic differences in metadata and field content while remaining single-sample, standards-compliant VCF files (2). For more information about test input data, to access raw test result data, or to replicate all tests performed, see `https://github.com/ecrum19/vcf-rdfizer-testing`.

Testing Environment VCF-RDFizer automatically logs wall time, user time, CPU time, peak memory (via resident set size (RSS)), output directory size, and total triple counts, along with many other metrics, to support reproducible performance comparisons across datasets and hardware setups. All metrics are collected via bash scripts at tool run-time to ensure reproducibility and consistent measurement across all experiments. Output validation in this study included syntax checks and targeted inspection of variant, genotype, and metadata nodes to verify that semantic links and datatypes were generated as intended.

All performance experiments were conducted on a workstation running Ubuntu 24.04.4 LTS with an Intel(R) Xeon(R) Silver 4309Y CPU @ 2.80GHz (8 cores, 2.8 GHz), 33.6 GB RAM and a 210 GB SSD, Kernel version 6.8.0-45-generic. Software versions: Docker 28.2.2, Python 3.12.3.

Results

VCF-RDFizer - General Performance Observations

VCF TSV Intermediate Generation. VCF inputs were first transformed into TSV intermediates to provide a compatible logical source for RMLStreamer and enable consistent field-level extraction during mapping. Across all six datasets, TSV generation completed in 11.9–47.6 seconds with peak RSS of only 4.4–5.4 MB. Each input produced three TSV artifacts (records, header lines, and file metadata) totaling 554.7–2582.3 MB, corresponding to 1.09×–1.43× the compressed VCF input size but only 1.78–5.44% of the final RDF output size. Relative to RDF conversion, TSV generation accounted for just 0.7–7.1% of conversion wall time, indicating minimal computational overhead while providing a stable logical source for declarative mapping execution.

RDF Conversion. RMLStreamer conversions completed successfully with wall-clock times of 621.0–1693.5 seconds (10.4–28.2 minutes) and peak RSS of 1546.3–2027.3 MB. Each dataset produced eight partitioned RDF outputs with well-balanced triple counts (e.g., 20.91–21.08 million triples per partition for 0GOOR_HG002), indicating stable parallel execution.

VCF-RDFizer was executed in *batch* output mode, which preserves partitioned RDF outputs for compression. Reported compression runtimes therefore represent dataset-level totals across all partition files rather than the cost of compressing a single aggregated RDF file.

Table 2 summarizes conversion results for six VCF datasets. Inputs ranged from 387.9–2076.8 MB and produced 25,969.3–77,729.7 MB of RDF, corresponding to semantic expansion factors of 20.0×–80.4× relative to compressed VCF input. Outputs contained 147.71–457.41 million triples, demonstrating that the workflow scales across substantially different dataset sizes. Overall, conversion remained tractable on commodity workstation hardware, although memory usage was non-trivial. Under this batch configuration, general-purpose compression required minutes, whereas HDT generation dominated end-to-end runtime.

RDF Output Compression. Compression results are summarized in Table 3. All methods substantially reduced RDF size but differed in compression ratio, runtime, and memory requirements.

HDT Compression. HDT reduced RDF outputs to 1876.3–5359.7 MB (92.62–94.68% reduction). However, it required the longest runtimes (64720.0–627977.0 seconds; 18.0–174.4 hours) and the highest memory usage (2566.1–6447.8 MB RSS). Across datasets, HDT accounted for 97.3% of total wall time. Runtime varied by 563257.0 seconds (156.5 hours), corresponding to a 9.7× increase between datasets *60820188474283* and *NG1N86S6FC*; the latter alone contributed 60.7% of total HDT processing time. User and wall times were nearly identical (99.9%), indicating a strongly CPU-bound stage consistent with HDT dictionary and index construction overhead.

Gzip Compression. Gzip reduced RDF size to 883.2–2800.3 MB (96.14–97.17% reduction) with runtimes of 183.0–577.0 seconds and peak RSS of ∼4 MB. These results confirm gzip as a fast, low-memory archival compression baseline.

Brotli Compression. Brotli produced smaller outputs than gzip for all datasets (480.9–1592.7 MB; 97.56–98.46% reduction) but required longer runtimes (276.0–868.0 seconds; 4.6–14.5 minutes). Peak RSS remained modest at 47.0–51.1 MB, indicating that the additional cost is primarily CPU time. Brotli therefore provides improved compression ratios with moderate runtime overhead while maintaining low memory requirements.

Compound Compression. Applying general-purpose compression after HDT produced the smallest artifacts. HDT+gzip reduced outputs to 215.8–682.1 MB (98.69–99.31% reduction) with an additional 26.0–87.0 seconds of runtime and negligible memory overhead (∼4 MB RSS). HDT+brotli produced the smallest files overall (182.0–682.4 MB; 98.84–99.42% reduction) with an additional 64.0–195.7 seconds of runtime and 79.1–110.2 MB peak RSS. Overall, HDT+gzip provided the fastest secondary compression, whereas HDT+brotli consistently achieved the smallest final outputs.

Input file	Time (s)					Size (MB)				
	HDT	Gzip	Brotli	HDT+Gz	HDT+Br	HDT	Gzip	Brotli	HDT+Gz	HDT+Br
0G00R_HG002	70,915.0	192.0	276.0	26.0	64.0	2,102.6	883.2	480.9	215.8	182.0
60820188474283	64,720.0	237.0	348.0	40.0	96.0	1,887.4	1,189.2	731.4	357.6	291.6
NG131FQA1I	92,266.0	183.0	328.0	38.0	106.0	1,876.3	974.2	614.0	320.0	293.2
60820188475559	87,931.0	249.0	366.0	40.0	99.3	2,020.6	1,271.7	775.8	371.0	318.9
NB72462M	91,409.0	204.0	350.0	40.0	117.0	1,975.9	1,032.6	652.8	349.6	311.5
NG1N86S6FC	627,977.0	577.0	868.0	87.0	195.7	5,359.7	2,800.3	1,592.7	682.1	682.4

Table 3 Compression statistics per input file. Times are wall-clock seconds; sizes are decimal MB.

Discussion

VCF-RDFizer provides a reproducible pathway for converting heterogeneous VCF datasets into structured RDF representations using a public semantic vocabulary and modular declarative mappings. Benchmarking indicates that VCF-to-RDF conversion is practical across the tested scale: input files of 387.9–2076.8 MB were converted in 10.4–28.2 minutes with peak RSS of approximately 1.5–2.0 GB and output RDF file sizes of 26–78 GB. For usage scenarios where storage efficiency is a concern, general-purpose compression (gzip or Brotli) can be applied to the raw RDF outputs with minimal runtime and memory overhead, while HDT compression provides additional functionality of compressed data at the cost of substantially longer runtimes and higher memory usage. In practice, the primary operational constraint for users is memory availability per concurrent conversion job rather than CPU capacity.

As illustrated by file *NG1N86S6FC*, increases in input file size directly increase processing time and memory usage for both conversion and compression steps. We suspect that, based on these results, multi-sample records would further increase computational, time, and memory complexity of VCF-RDFizer. Consequently, large multi-sample datasets may require staged processing strategies, additional hardware requirements, or parallelization approaches to mitigate these challenges.

To explicitly state the current capabilities of our tool, Table 4 summarizes which VCF constructs are fully supported, partially supported, or not yet implemented in VCF-RDFizer (v1.1). Core VCF record fields are fully supported while heterogeneous elements such as complex INFO annotations and alternate allele representations require additional modeling work.

Semantic Representation of VCF Challenges

Because VCF encodes both biological observations and file-structure semantics, representation in RDF requires a vocabulary that explicitly models the structural elements of the format itself. Existing ontologies capture important aspects of genomic variation, including FALDO for genomic coordinates, the Sequence Ontology (SO) (17) for variant types, GENO for genotype modeling (7), and HERO for clinically oriented genomic semantics (24), but none fully represent the structural semantics of the VCF specification.

To address this gap, we developed the publicly available VCF-RDFizer vocabulary (https://w3id.org/vcf-rdfizer/vocab#), which provides a lightweight semantic layer aligned with the structural components of the VCF specification while remaining interoperable with existing genomic ontologies. Future work will extend the vocabulary to better capture heterogeneous INFO and FORMAT field semantics, evaluate our modeling approach on multi-sample VCF datasets, and introduce additional reasoning or validation constraints (e.g. SHACL rules) to improve semantic consistency. Although the current RML mappings theoretically support multi-sample VCF files, our evaluation focused on single-sample datasets to establish baseline feasibility.

The Published VCF-RDFizer Software Tool

Transformation Architecture and Performance. Our evaluation demonstrates that RML provides a principled framework for converting VCF data into RDF. The declarative mapping model separates semantic intent from procedural parsing logic, allowing transformation rules to be versioned and reused across datasets. Results indicate that using our default RML ruleset, RMLStreamer can effectively convert VCF data into RDF with reasonable performance on commodity hardware. Across the evaluated datasets, memory usage remained below 2 GB and runtime scaled predictably with input size, demonstrating that the approach is practical for typical workstation environments.

The current pipeline includes a lightweight preprocessing step that converts VCF files into tabular intermediates compatible with existing RML engines. Because VCF is not yet supported as a native logical source within RML, this step enables stable field extraction and declarative mapping execution while adding minimal overhead. Future extensions to the RML ecosystem could support VCF as a native logical source, eliminating this preprocessing stage.

Overall, these results confirm that VCF-RDFizer successfully performs the intended semantic lifting of VCF datasets into RDF. The primary practical limitation observed in this study is not the conversion process itself but the substantial expansion in output size inherent to RDF representations. This increase in data volume motivates the use of compression strategies to make semantic genomic datasets practical for storage and distribution.

Compression Strategies for Semantic Genomic Data. General-purpose compressors provide effective baseline solutions: gzip with minimal memory overhead and 96.14–97.17% size reduction, and Brotli with stronger compression (97.56–98.46%) at moderately increased runtimes. For workflows requiring queryable compressed data, HDT (Header–Dictionary–Triples) provides a specialized representation combining compression with indexing (18). However, in the current implementation HDT incurred substantially longer wall times due to index construction.

The rapid runtime increase observed for larger inputs (e.g., *NG1N86S6FC*) is consistent with the HDT build procedure and the `hdt-cpp` implementation. HDT generation requires whole-dataset dictionary construction and sorting of ID-triples into queryable BitmapTriples structures, while the current tool materializes triples in memory before sorting. As input size grows, these steps scale superlinearly and can introduce significant memory pressure, leading to substantial runtime inflation. To mitigate these costs, future work will explore alternative construction strategies such as batching RDF

VCF component / capability	Supported now	Partially supported	Not supported	Notes / limitations
Core fields (CHROM, POS, ID, REF, ALT, QUAL, FILTER)	✓			Covered by core mappings and deterministic IRI templates
INFO fields		✓		Supported in tested datasets; high variability in semantics requires ongoing refinement
FORMAT fields		✓		Supported in tested datasets; complexity increases in multi-sample VCF files
Sample/genotype fields		✓		Supported in evaluated single-sample inputs; un-tested for multi-sample VCF files
HEADER metadata		✓		Full coverage in tested datasets; complete semantic coverage remains challenging
Multi-sample support		✓		Multi-sample modeling theoretically supported; benchmarks are only single-sample VCF files
Structural variants		✓		Complex alternate-allele data supported; semantic tagging and validation as future work
Symbolic alleles (e.g., `<DEL>`, `<INS>`)			✓	Not explicitly supported or evaluated here; planned within future extensions.
Provenance/policy annotations			✓	Discussed as future governance capability; not yet implemented in the conversion pipeline

Table 4 Current VCF-RDFizer tool capabilities by VCF field/component, based on the implementation and evaluation scope reported in this study.

inputs and merging intermediate HDT files and indexes using tools like HDTCat (16).

At present, these results suggest a use-based workflow choice: gzip for fast archival compression, Brotli for maximal size reduction, and HDT when compressed datasets must remain queryable despite the higher construction cost. Other semantic compression approaches, such as Jelly (6), may offer additional optimization opportunities in future work.

Tool Availability and Community Impact. Following best practices for research software publication (21), VCF-RDFizer is distributed as open-source software with public version control and synchronized releases across GitHub, PyPI, Conda-Forge, and DockerHub under the MIT License. Public issue tracking, release notes, and a documented maintenance plan support transparency and long-term sustainability.

Implications for Semantic Genomic Workflows. This work demonstrates that VCF datasets can be lifted into graph-based representations with resource requirements compatible with typical bioinformatics workflows. RDF enables expressive queries spanning variants, genotypes, samples, and linked biomedical resources. When combined with HDT storage and modern SPARQL engines (29), these representations support compressed yet queryable variant datasets.

Stable IRIs and ontology-aligned predicates improve interoperability with biomedical knowledge graphs, enabling direct connections between variant observations and phenotype or pharmacogenomic resources. Graph-based models also support flexible governance mechanisms, allowing access-control metadata to be attached to individual entities or graph partitions using standards such as ODRL (20). Together, these capabilities position semantic VCF representations as infrastructure for integrating genomic datasets within broader biomedical knowledge graph ecosystems.

Impact and Future Work

This work demonstrates that VCF datasets can be systematically transformed into RDF using declarative RML semantic mappings. More importantly, we show that traditionally static file-based genomic variant data can be reliably converted to structured knowledge graphs, capable of linking, semantic querying, and governance-aware data sharing.

Future work will focus on extending semantic coverage of heterogeneous VCF structures (particularly INFO, FORMAT, GENOTYPE, and HEADER fields), improving performance for multi-sample datasets, and implementing VCF as a native logical source within the RML ecosystem. Additional evaluation of SPARQL querying over serialized and HDT-compressed outputs will also be required to assess practical query performance at genomic scale.

Ultimately, VCF-RDFizer represents a step toward semantically interoperable genomic data infrastructures in which variant datasets can be linked, queried, and governed as part of a broader biomedical knowledge graph ecosystem.

Conflicts of interest

None declared.

Funding

This work is supported by the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N).

Data availability

All underlying data for this study are available at `https://github.com/ecrum19/vcf-rdfizer-testing`.

Author contributions statement

E.C. and G.E. conceived the tool and the experiments. E.C. implemented the conversion workflow and executed the experiments. E.C. and R.T. analyzed the results. E.C. wrote the manuscript draft. E.C., R.T., G.E., and B.B. reviewed and edited the manuscript.

Acknowledgments

All benchmarking computation was performed using the resources of IDLab at Ghent University.

References

1. Personal Genome Project, 2005.
2. The Variant Call Format Specification, October 2024.
3. NIST Genome in a Bottle, November 2025.
4. yaml/yaml-spec, November 2025. original-date: 2009-04-01T20:52:05Z.
5. Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, et al. Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems*, 37(1):1–30, January 2019.
6. Julián Arenas-Guerrero and Sebastián Ferrada. Cottas: Columnar triple table storage for efficient and compressed rdf management. In *The Semantic Web – ISWC 2025*. Springer, 2025.
7. Michael Ashburner, Catherine A. Ball, Judith A. Blake, et al. Gene Ontology: tool for the unification of biology. *Nature Genetics*, 25(1):25–29, May 2000.
8. François Belleau, Marc-Alexandre Nolin, Nicole Tourigny, et al. Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics*, 41(5):706–716, October 2008.
9. Arto Bendiken, Raoul J.P. Bonnal, and Francesco Strozzi. ruby-rdf/rdf-vcf: Rdf vcf parser, November 2025.
10. Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, et al. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
11. Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, et al. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.
12. Anna Breit, Simon Ott, Asan Agibetov, et al. OpenBioLink: A benchmarking framework for large-scale biomedical link prediction. 2019. Publisher: arXiv Version Number: 2.
13. Richard Cyganiak, David Wood, and Markus Lanthaler. Rdf 1.1: Concepts and abstract syntax, February 2014.
14. Petr Danecek, James K Bonfield, Jennifer Liddle, et al. Twelve years of samtools and bcftools. *GigaScience*, 10(2):giab008, 2021.
15. Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.
16. Dennis Diefenbach and José M. Giménez-García. Hdtcat: Let's make hdt generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33. Springer, 2020.
17. Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, et al. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.
18. Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, et al. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.
19. Steve Harris, Andy Seaborne, and Eric Prud'hommeaux. SPARQL 1.1 query language, March 2013.
20. Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.
21. Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, et al. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.
22. Binglan Li, Katrin Sangkuhl, Karl Keat, et al. How to run the pharmacogenomics clinical annotation tool (pharmcat). *Clinical Pharmacology & Therapeutics*, 113(5):1036–1047, 2023.
23. Pierre Lindenbaum. JVarkit: java-based utilities for Bioinformatics. 5 2015.
24. Laura Menotti, Mirco Cazzaro, Manuel Rueda, et al. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
25. Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, et al. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.
26. Sitt Min Oo, Gerald Haesendonck, Ben De Meester, et al. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, et al., editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.
27. SIB Swiss Institute of Bioinformatics RDF Group Members, Adrian Altenhoff, Amos Bairoch, et al. The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research*, 52(D1):D44–D51, January 2024.
28. Erika Souche, Sergi Beltran, Erwin Brosens, et al. Recommendations for whole genome sequencing in diagnostics for rare diseases. *European Journal of Human Genetics*, 30(9):1017–1021, September 2022.
29. Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, et al. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255. Springer International Publishing, 10 2018.
30. Eelke Van Der Horst, Deepak Unni, Femke Kopmels, et al. Bridging Clinical and Genomic Knowledge: An Extension of the SPHN RDF Schema for Seamless Integration and FAIRification of Omics Data. December 2023.
31. Alex H. Wagner, Lawrence Babb, Gil Alterovitz, et al. The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics*, 1(2):100027, November 2021.
32. Alex H. Wagner, Reece Hart, Larry Babb, et al. ga4gh/vrs: 2.0.1, 2025.
33. World Wide Web Consortium (W3C). Linkeddata.